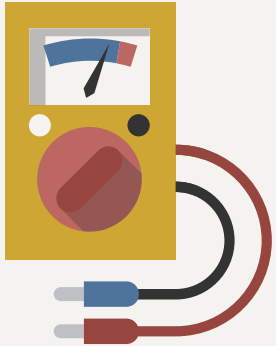
ECE532 FINAL DEMO PRESENTATION

FPGA CIRCUIT SIMULATOR



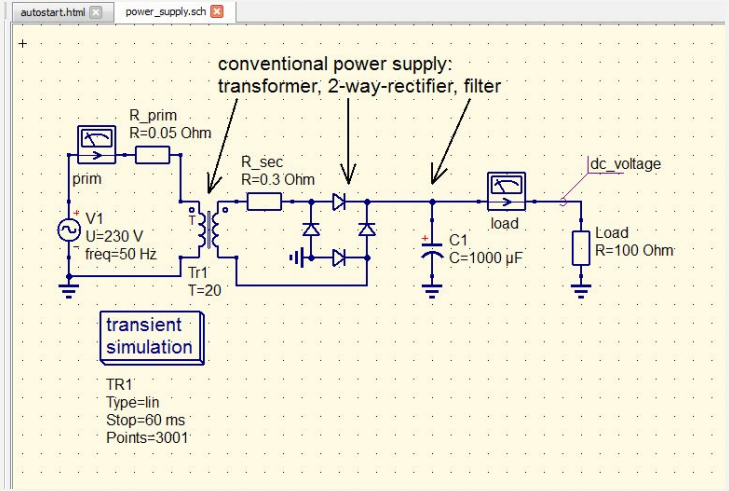
DAVID MONTES DE OCA | SAHIL AMIN
IBRAHIM SWEIDAN | ANDREW RADKE

Agenda for Presentation:

1. High Level Project Description
2. Initial Goal + Block Diagram of System
3. Final Implemented Project + Block Diagram of System
4. Explanation of major components (VGA Display, Custom IPs, Xilinx IPs, UART)
   – detailed explanation of the compute/logic complexity of any custom hardware blocks
   – explanation of software flow in processor system (interrupts, event loop, etc.)
   – explanation of bus configuration, dataflow schema, and data storage (if applicable)
5. Final Computed Complexity Score Table
6. Future Improvements (if we had more time)
7. Demo

TOPIC OVERVIEW

Existing spice simulators, such as Simulation Program with Integrated Circuit Emphasis (SPICE), are based primarily on software

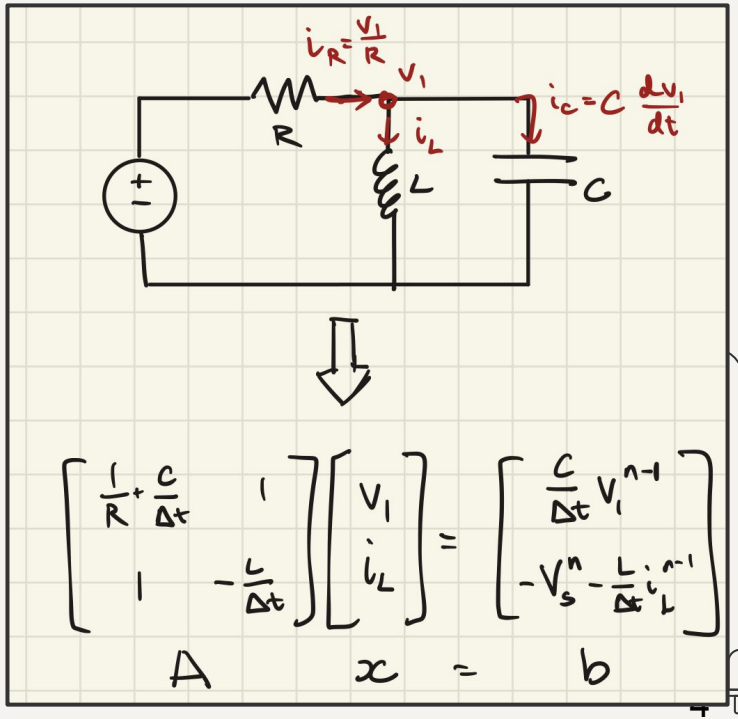


FPGA-based systems can exploit parallel hardware to theoretically improve performance

MODIFIED NODAL ANALYSIS

Maps a circuit into a linear system using KCL, ***Ax* = *b***.

- ***A*** matrix: nodal matrix of R,L,C components
- x vector: unknown voltages at each node
- ***b*** vector: voltage/current sources in system.



INITIAL GOAL

Supported simulation types

DC Operating point

Parameter Sweep

AC Steady State

AC Frequency Sweep

Transient

Components supported

- ☐ Voltage Sources
- ☐ Current sources
- ☐ Resistors
- ☐ Capacitors
- ☐ Inductors

FINAL ACHIEVEMENTS

Supported simulation types

~~DC Operating point~~

Parameter Sweep

AC Steady State

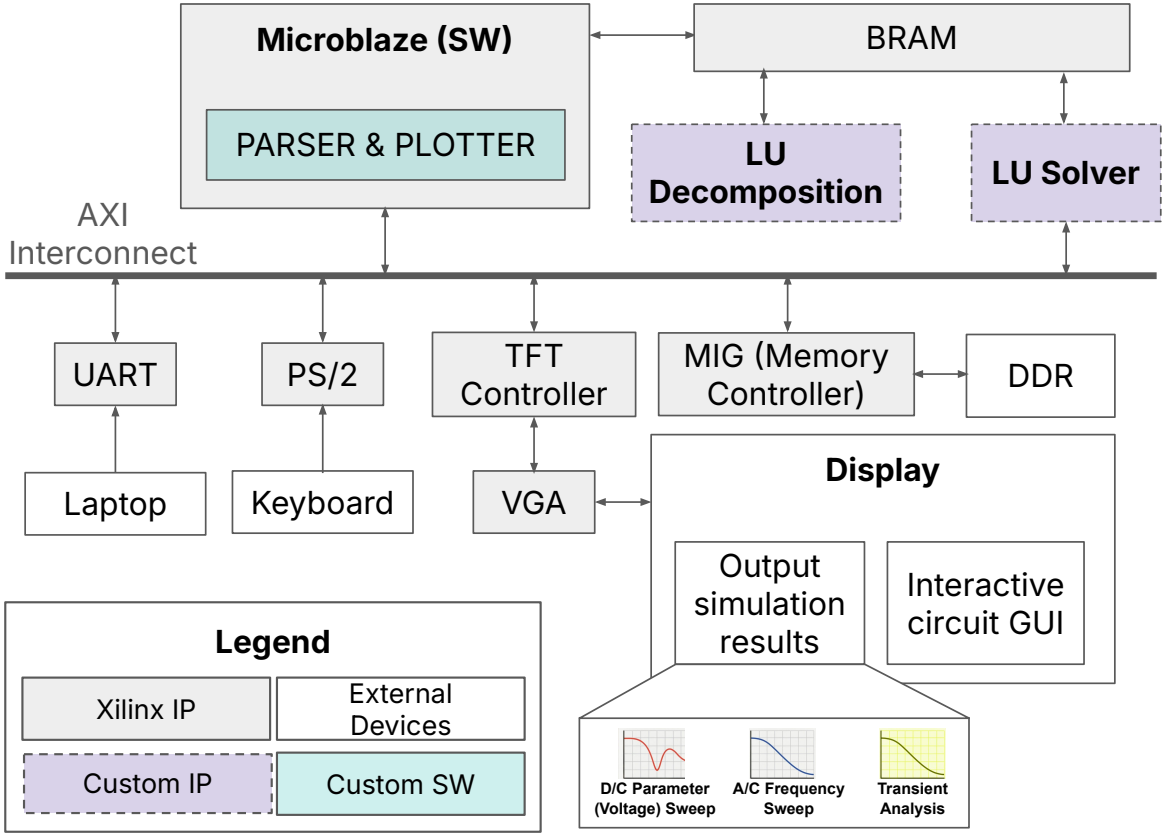
AC Frequency Sweep

~~Transient~~

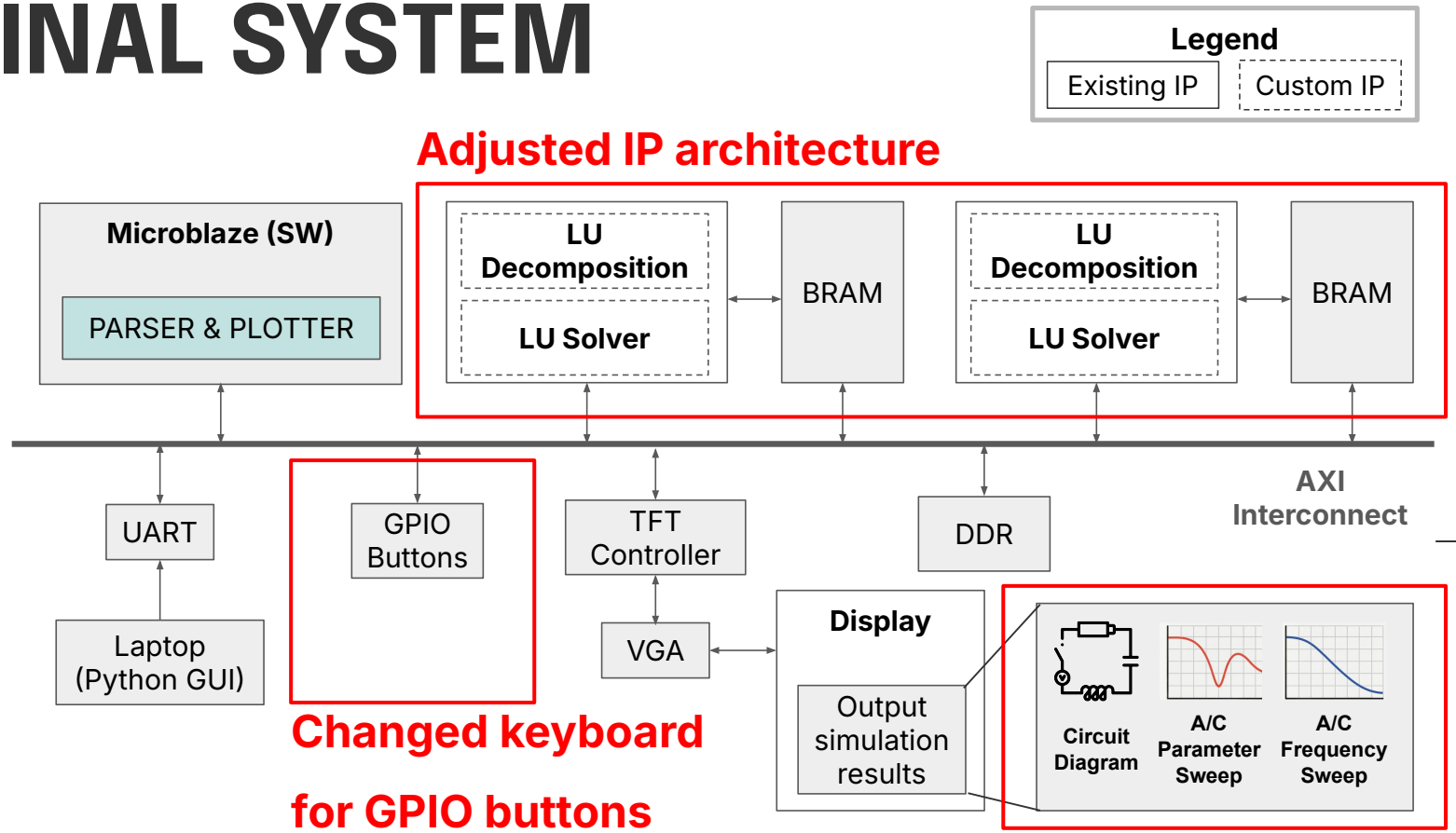
Components supported

- ☐ Voltage Sources
- ☐ Current sources
- ☐ Resistors
- ☐ Capacitors
- ☐ Inductors

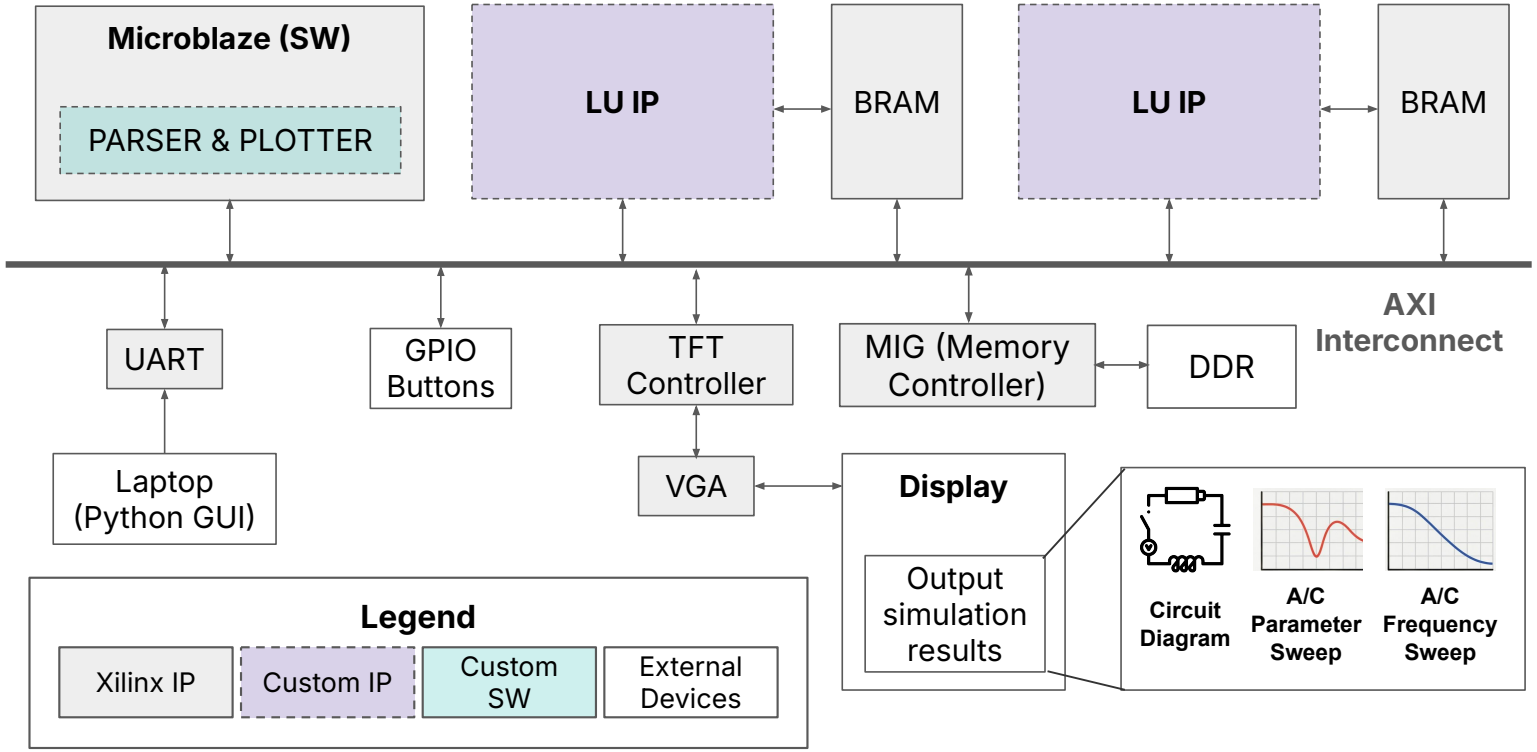
INITIAL SYSTEM



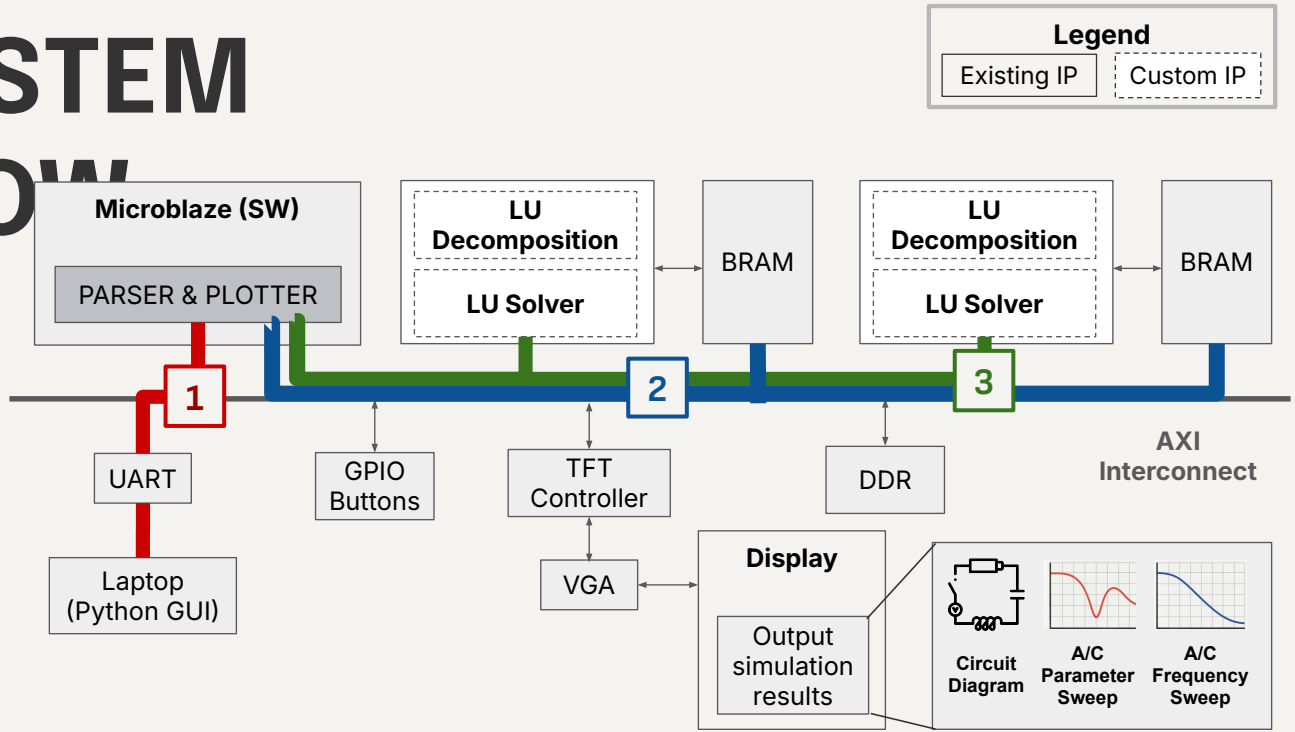
FINAL SYSTEM



FINAL SYSTEM

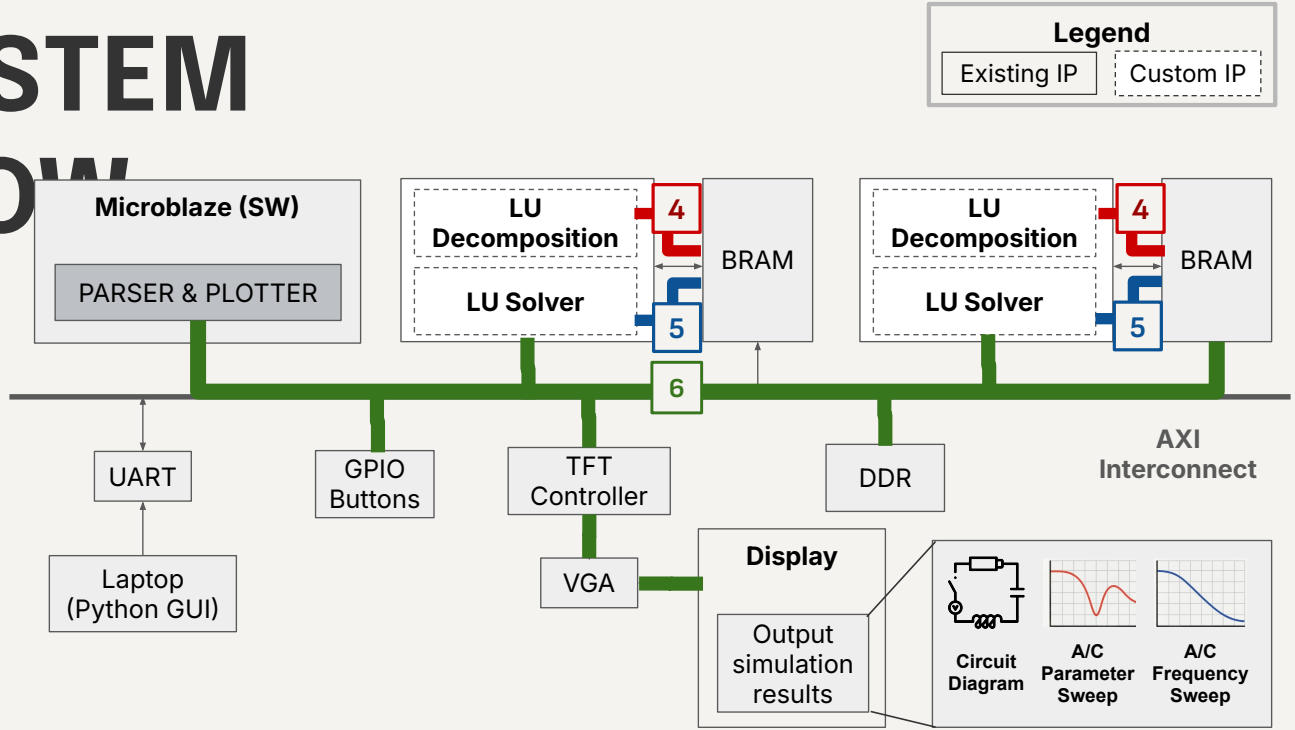


SYSTEM FLOW



1. Input SPICE code via laptop GUI
2. Write ***A*** and ***b*** vectors from Parser into each BRAM via AXI4
3. Write to AXI4-LITE registers on Custom IP to start processing

SYSTEM FLOW

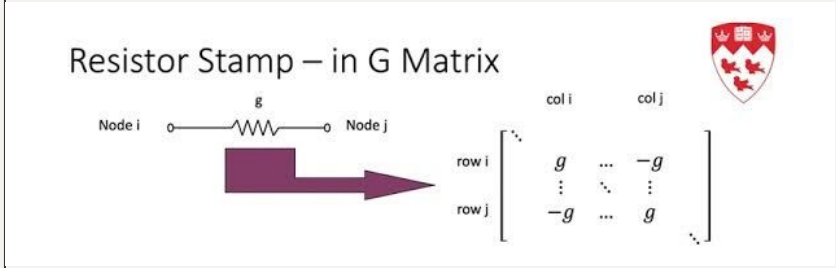


4. LU Decomp reads ***A*** and ***b***, uses it to produce ***L***, ***U***.
   [Only done once for parameter sweep per core, as L,U are reused]

5. Solver uses ***L***,***U*** to compute x, and write to BRAM.

6. Microblaze reads x from BRAM, and writes to VGA display.

SW OVERVIEW: MNA Code



Roni Khazaki, McGill University

```
# header
.ac 10M
.node v1
.sweep R1 1 16.5 0.5
# code
V1 vdd 0 1.0
R1 vdd v1 10
R2 v1 v2 5
R3 v2 v3 3
C1 v1 0 5n
C2 v2 0 6n
C3 v3 0 7n
```

Stage 1: Receive Netlist

- Receive SPICE text from UART
- Put string into BRAM

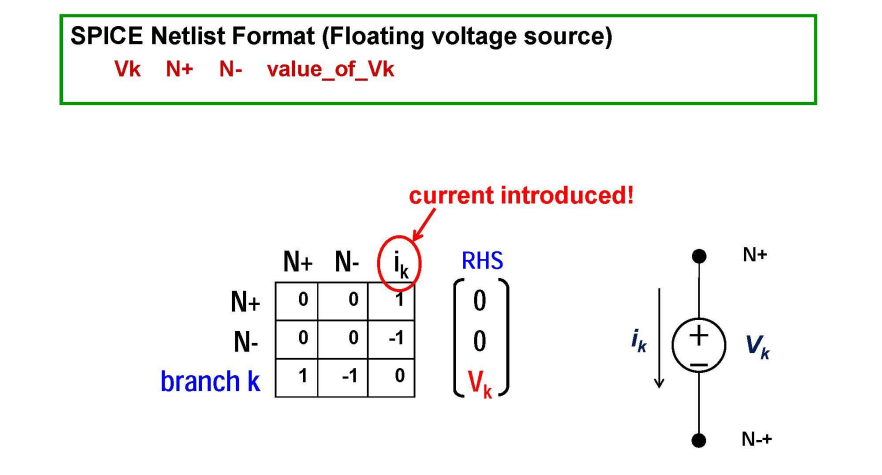
Stage 2: Parse Netlist

- Read string from BRAM
- Convert string to netlist

Stage 3: MNA

- Make matrix of size n+m
- Compute admittance and stamp matrix for each element
- Assign nodes to sources

SW OVERVIEW: MNA Code



Guoyong Shi, Shanghai Jiao Tong University

Stage 4: BRAM Write

- Write matrix to solver block and start

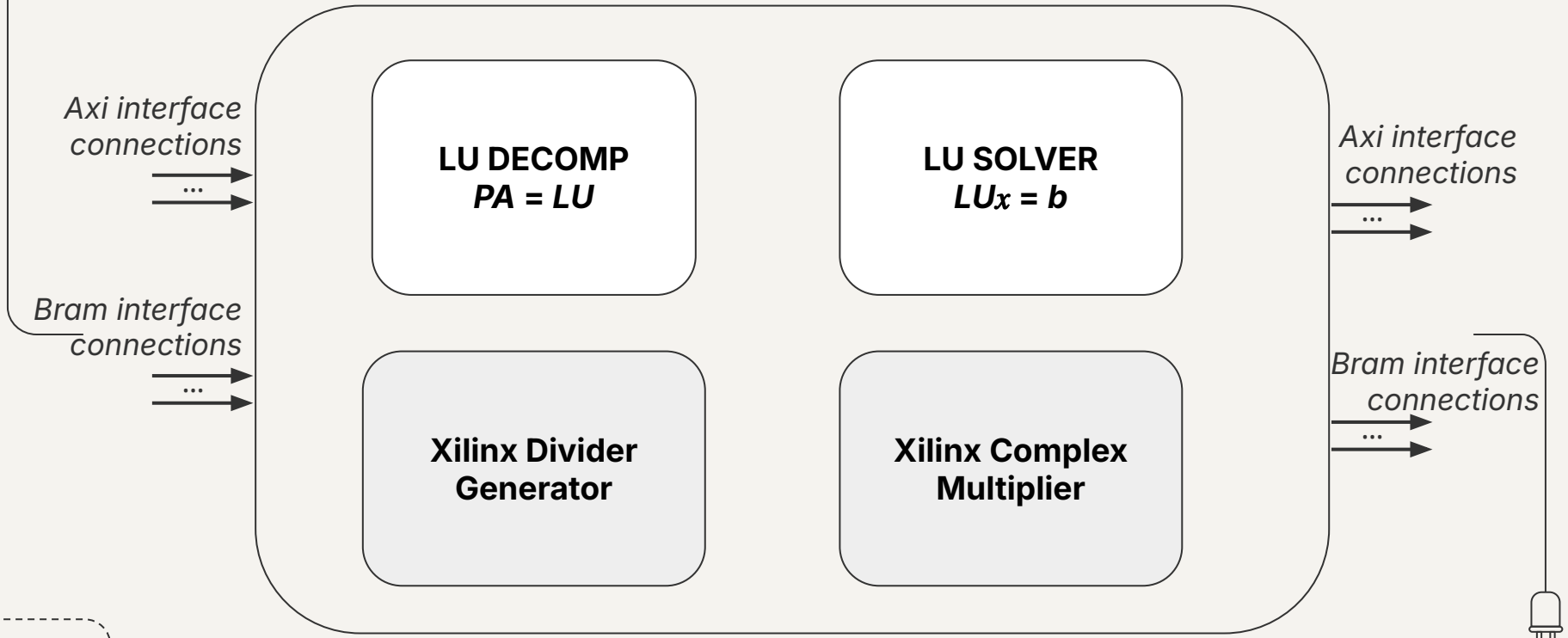
Stage 5: Repeat

- Repeat stage 3 and 4 for each point in sweep

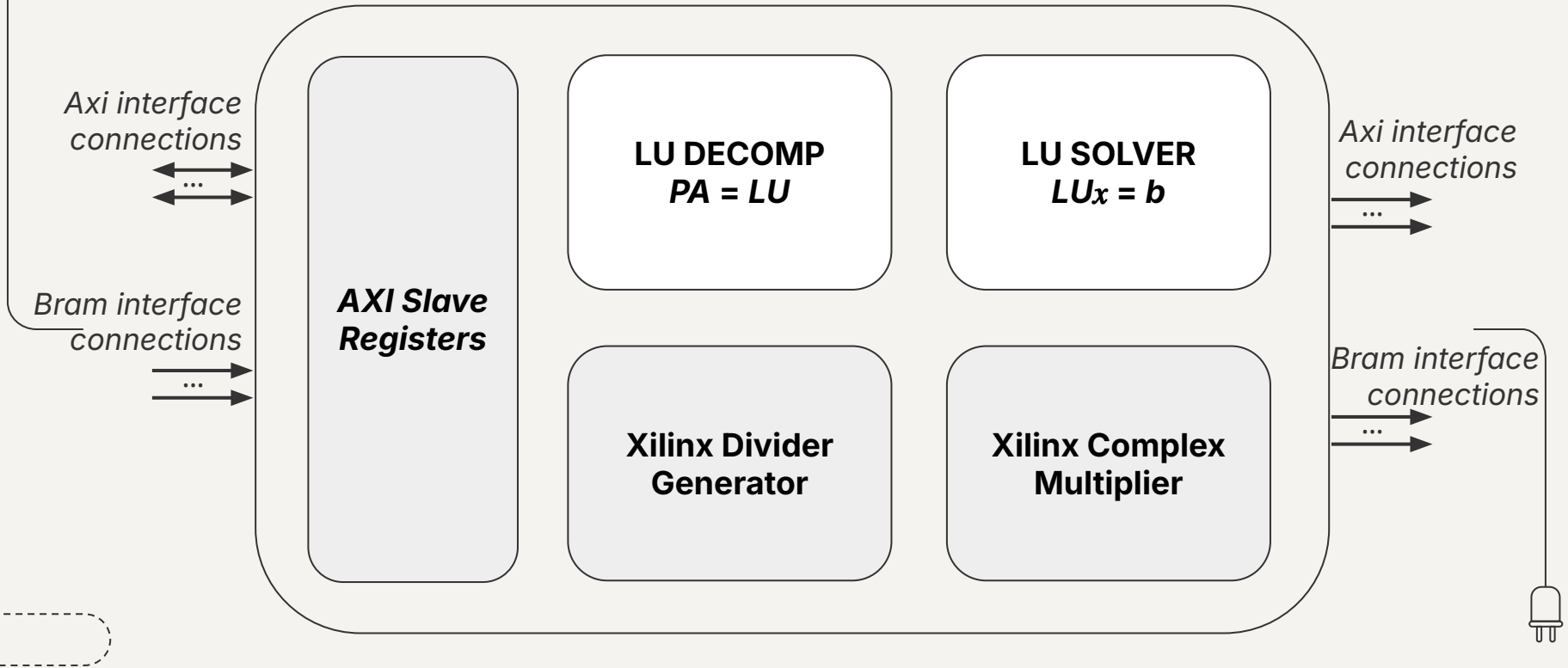
Stage 6: Finish

- Write all results to BRAM
- Send results back through UART

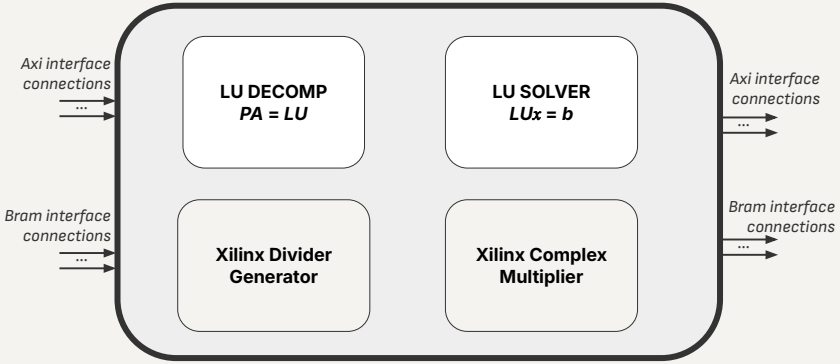
IP OVERVIEW: LU TOP



IP OVERVIEW: LU TOP



IP OVERVIEW: LU TOP



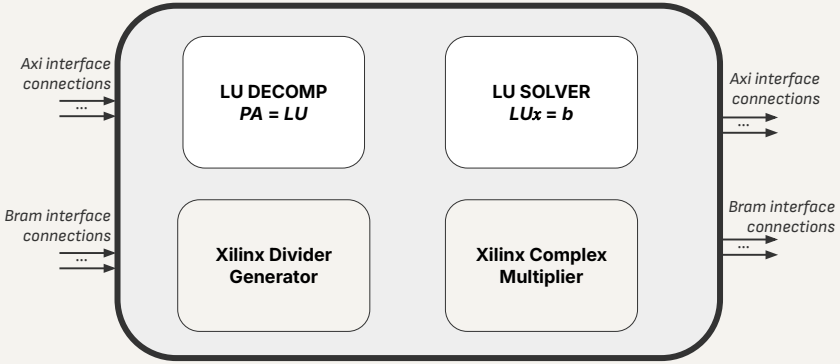
Inputs:

1. From microblaze through AXI
   a. ***A*** matrix base address in BRAM
   b. ***b*** vector base address in BRAM
   c. $\boldsymbol{x}$ output base address in BRAM
   d. Start pulse [Full / Reuse Mode]

Outputs:

1. Writes $\boldsymbol{x}$ (solved unknowns) directly into BRAM
2. Changes AXI status reg to indicate done/fail

IP OVERVIEW: LU TOP



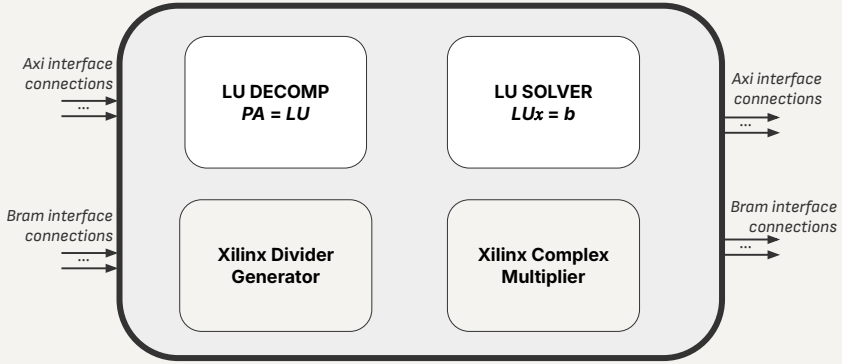
Number Format:

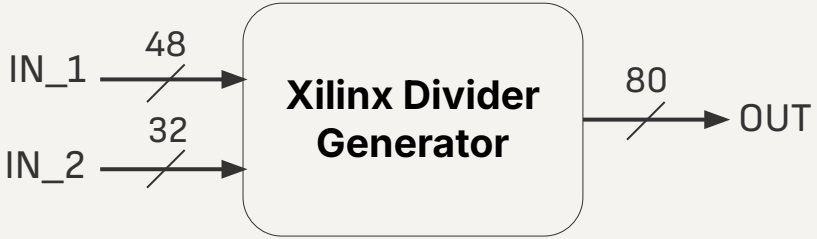
- Q16.16 format [for real and imaginary]
  This is the input and final output format.

Complex Multiplier: (a + bi)(c + di)

- IN_1 = [imag_in1, real_in1]
  IN_2 = [imag_in2, real_in2]
  64 bits per input [Q16.16 × 2]
- OUT = [imag_out, real_out]
  128 bit output [Q32.32]
- We truncate this from Q32.32 → Q32.16 for accumulator.

IP OVERVIEW: LU TOP





Number Format:

- Q16.16 format [for real and imaginary]
  This is the input and final output format.

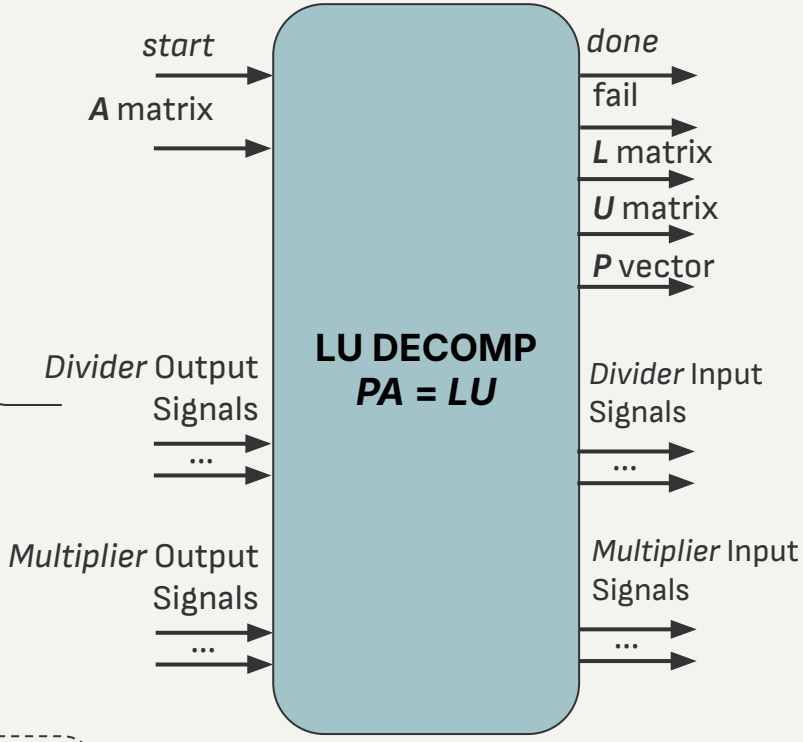
Divider Generator: a / b

- IN_1 : 48 bits for Q32.16 [shift 16 bits left]
  IN_2 : 32 bits for Q16.16

- OUT = 80 bits
  [quotient (48), remainder (32)]

  We care about bottom Q16.16 of quotient. (i.e. OUT[63:32])

IP OVERVIEW: LU DECOMPOSITION



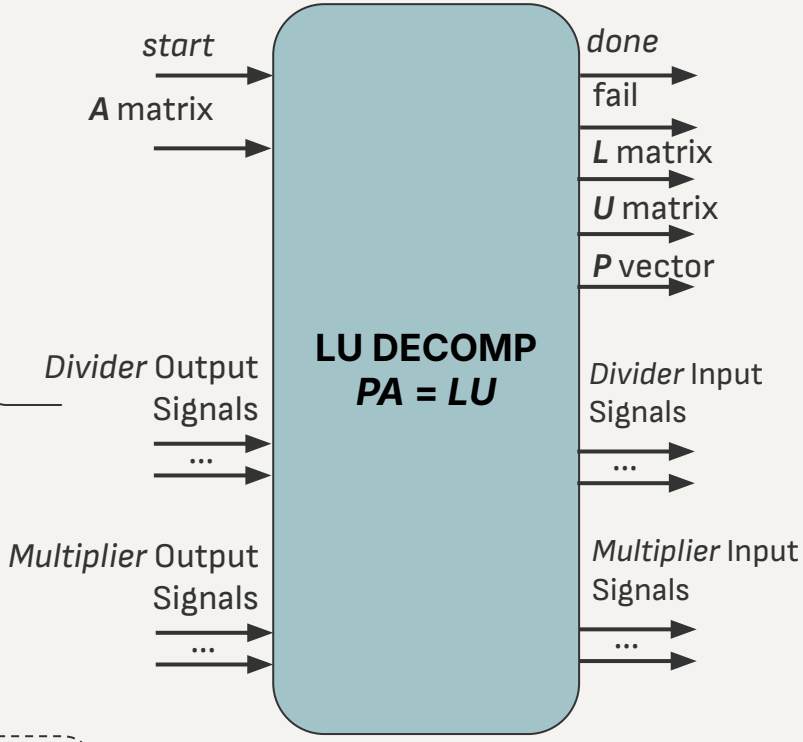
Inputs: A matrix *(from BRAM)*

Outputs: L,U,P

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 7 & 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 0 & -3 & -6 \\ 0 & 0 & 0 \end{bmatrix}$$

$$A = LU$$

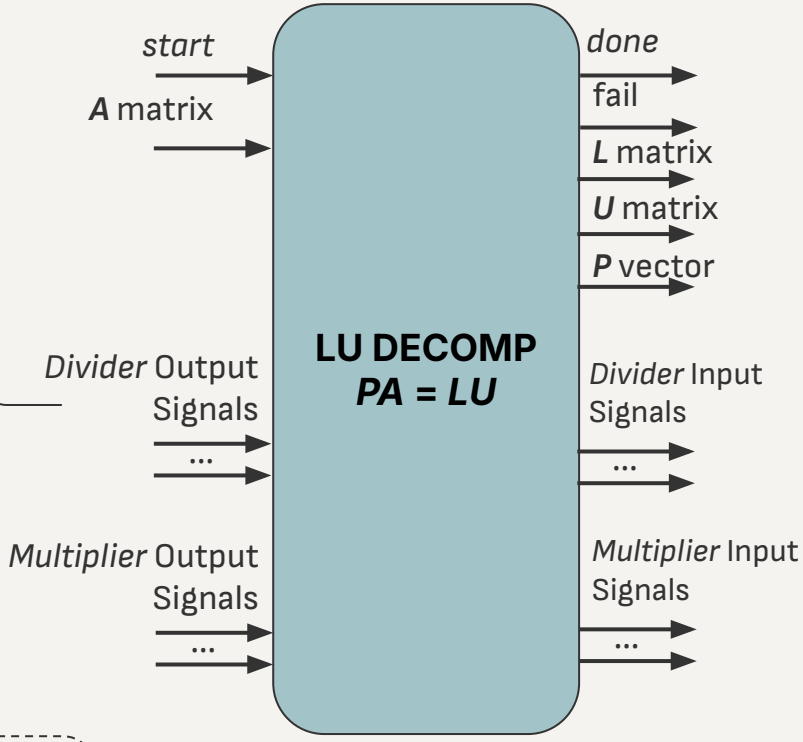
IP OVERVIEW: LU DECOMPOSITION



Algorithm:

1. **Pivot selection**
   a. At each column k, find row with largest A[r][k] for r > k
2. **Row swap**
   a. Swap that row into position k
3. **Division Factor**
   a. For each row i>k: L[i][k]=A[i][k]/A[k][k]
   b. This forms the L matrix

IP OVERVIEW: LU DECOMPOSITION

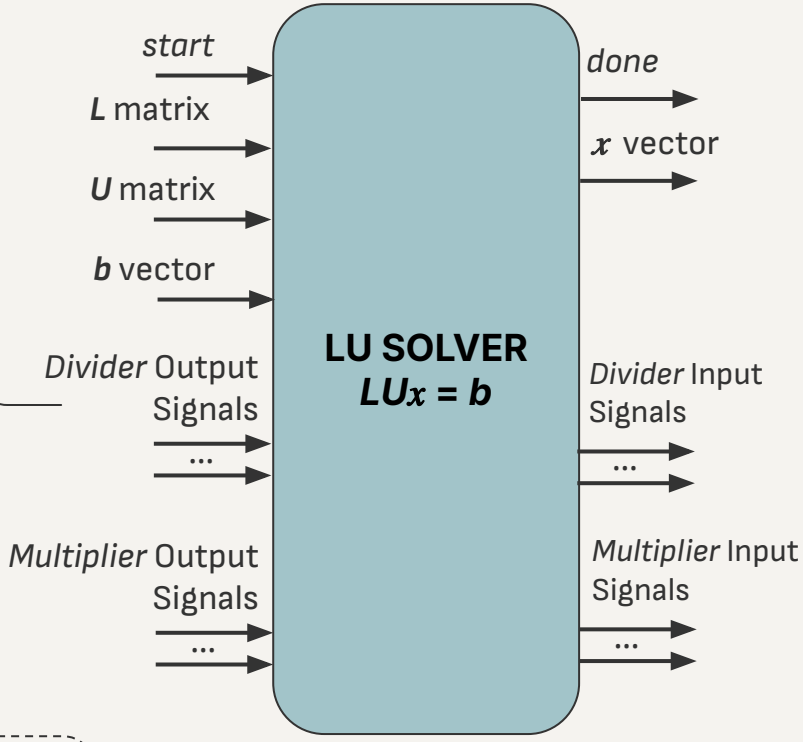


Algorithm (continued):

4. **Elimination**
   a. $A[i,j] \leftarrow A[i,j] - L[i,k] \cdot A[k,j]$
   b. This forces all elements under k to be 0

- After these steps are repeated for each pivot k:
  - ***L*** becomes lower triangular
  - ***A*** transforms into upper triangular
  - ***P*** encodes the row swaps

IP OVERVIEW: LU SOLVER



Inputs: L,U,b *(from LU Decomposition)*

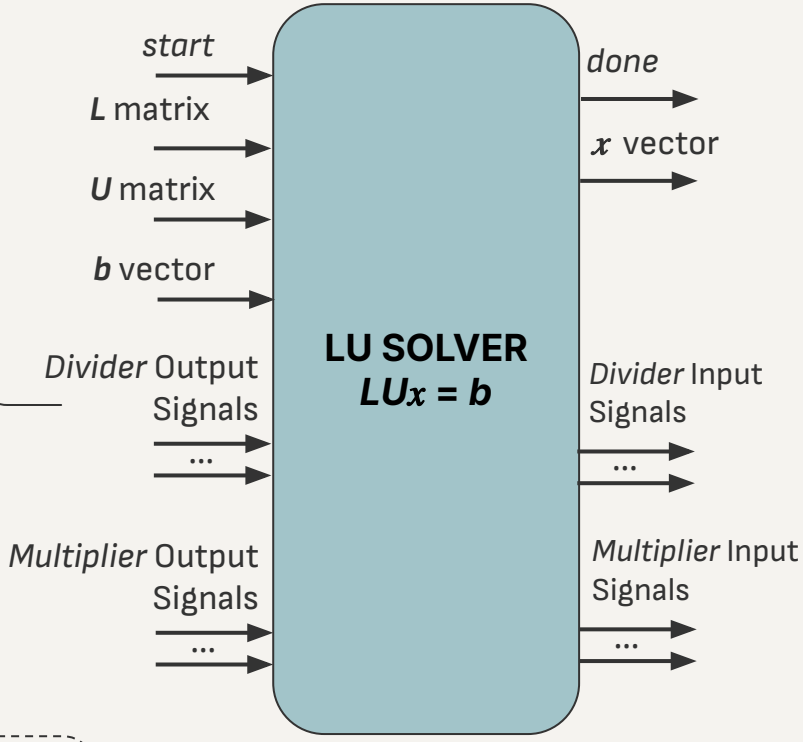
Outputs: x

Complex Number Arithmetic:

- <u>Complex Divide:</u> Xilinx Divider Generator IP, and staged combinational multiply and accumulate.

$$\frac{a+bi}{c+di} \Rightarrow \frac{(a+bi)\cdot(c-di)}{(c+di)\cdot(c-di)} \Rightarrow \frac{(a+bi)(c-di)}{c^2+d^2}$$

IP OVERVIEW: LU SOLVER



Stage 1: Forward Substitution

- Solve **L*y* = b** for ***y***

$$\begin{bmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

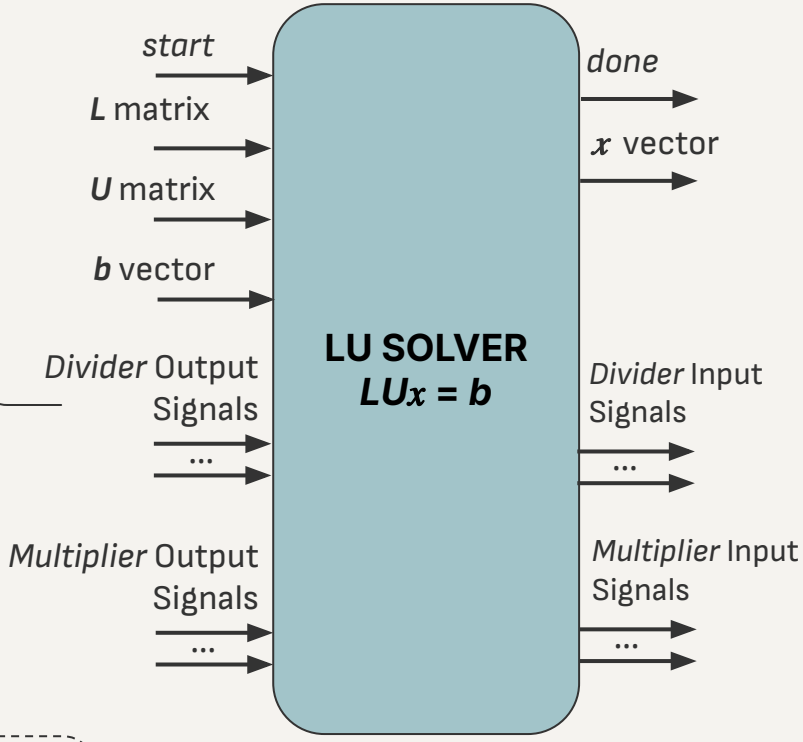
forward solve of y

$$L_{11} \cdot y_1 = b_1 \Rightarrow y_1 = b_1 / L_{11}$$

$$L_{21} y_1 + L_{22} \cdot y_2 = b_2 \Rightarrow y_2 = (b_2 - L_{21} y_1) / L_{22}$$

$$L_{31} \cdot y_1 + L_{32} \cdot y_2 + L_{33} \cdot y_3 = b_3 \Rightarrow y_3 = (b_3 - L_{31} y_1 - L_{32} y_2) / L_{33}$$

IP OVERVIEW: LU SOLVER



Stage 2: Backward Substitution

- Solve **U**x **= *y*** for x

$$\begin{bmatrix} U_{11} & U_{12} & U_{13} \\ 0 & U_{22} & U_{23} \\ 0 & 0 & U_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

backward solve of x

$$U_{33} \cdot x_3 = y_3 \Rightarrow x_3 = y_3 / U_{33}$$

$$U_{22} x_2 + U_{23} \cdot x_3 = y_2 \Rightarrow x_2 = (y_2 - U_{23} x_3) / U_{22}$$

$$U_{11} \cdot x_1 + U_{12} x_2 + U_{13} \cdot x_3 = y_1 \Rightarrow x_1 = (y_1 - U_{13} x_3 - U_{12} x_2) / U_{11}$$

SW OVERVIEW: VGA

Reads parsed netlist to draw circuit and highlight probed net

- Breaks circuit up into rows and columns
- Each “cell” contains either wire/ground/element
- Use obtained simulation parameter to highlight probed net

Supports frequency sweep bode plots and voltage sweep linear plots

- Uses stored values in BRAM and convert to dB/phase (if applicable)
- Newton-Raphson approximation and dB LUT for fast computations
- Dynamic axis labelling for voltage sweep linear plots only

SW OVERVIEW: VGA

Displays point information and allows traversal between points

- Left button to move left, right button to move right
- After plotting, event loop detects GPIO button presses with debouncing
- Centre button lets the laptop know the FPGA is ready to receive another spice file

COMPLEXITY POINTS TABLE

LU Decomposition IP	2
LU Solver IP	1
VGA output using MicroBlaze	0.75
Visualize meaningful results with a GUI	0.75
Software algorithm implemented on MicroBlaze	0.4
Transfer data from Desktop to FPGA using UART + MicroBlaze + python script	0.25
TOTAL	5.15

FUTURE IMPROVEMENTS

Number format constraints

- Higher precision fixed point or floating point
  - Floating point would require custom arithmetic IP blocks
- Increase intermediate registers to handle wider divides and multiplies

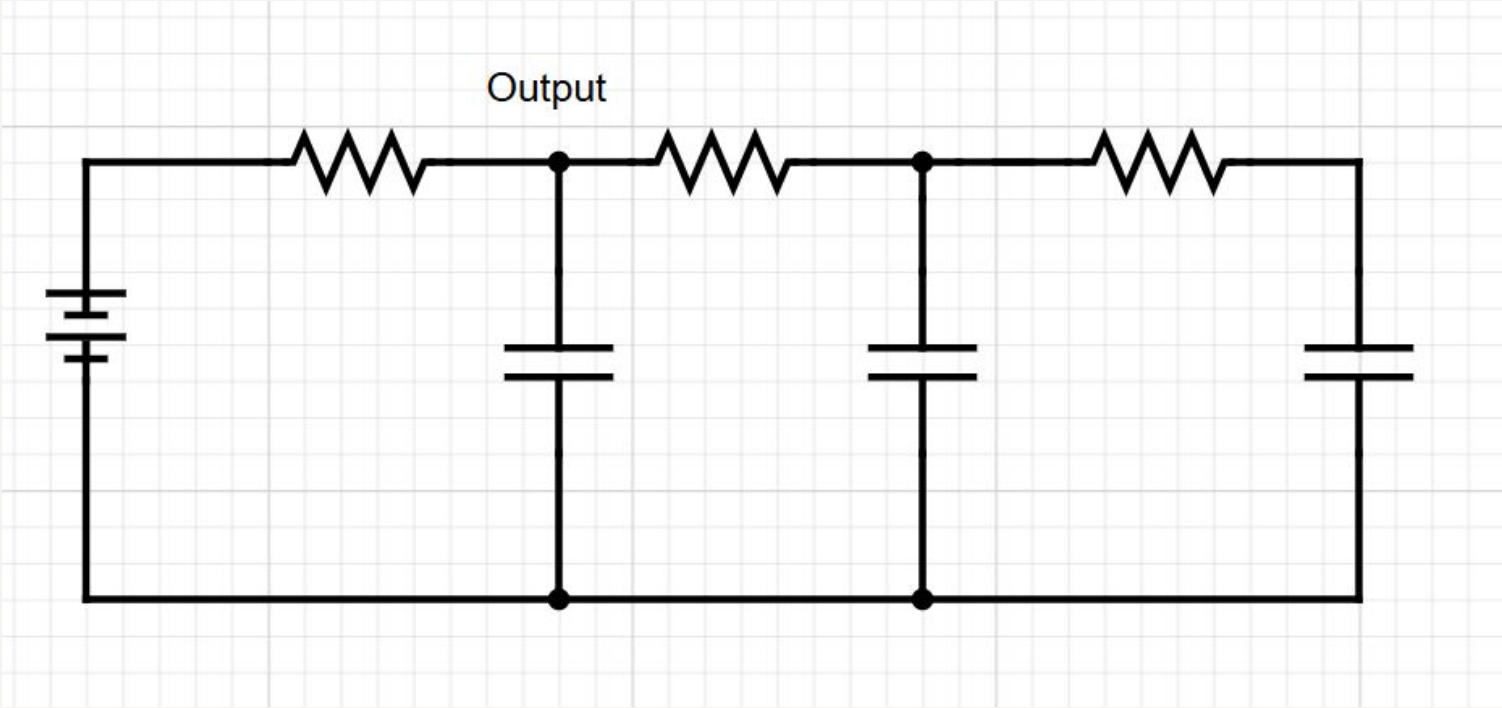
Improve parallel IP support

- Currently does not improve performance as expected
- Run microblaze at higher frequency and have CDC
  - Improve Fmax by using BRAM instead of distributed RAM and LUT registers to reduce congestion
  - Instruction and data caching

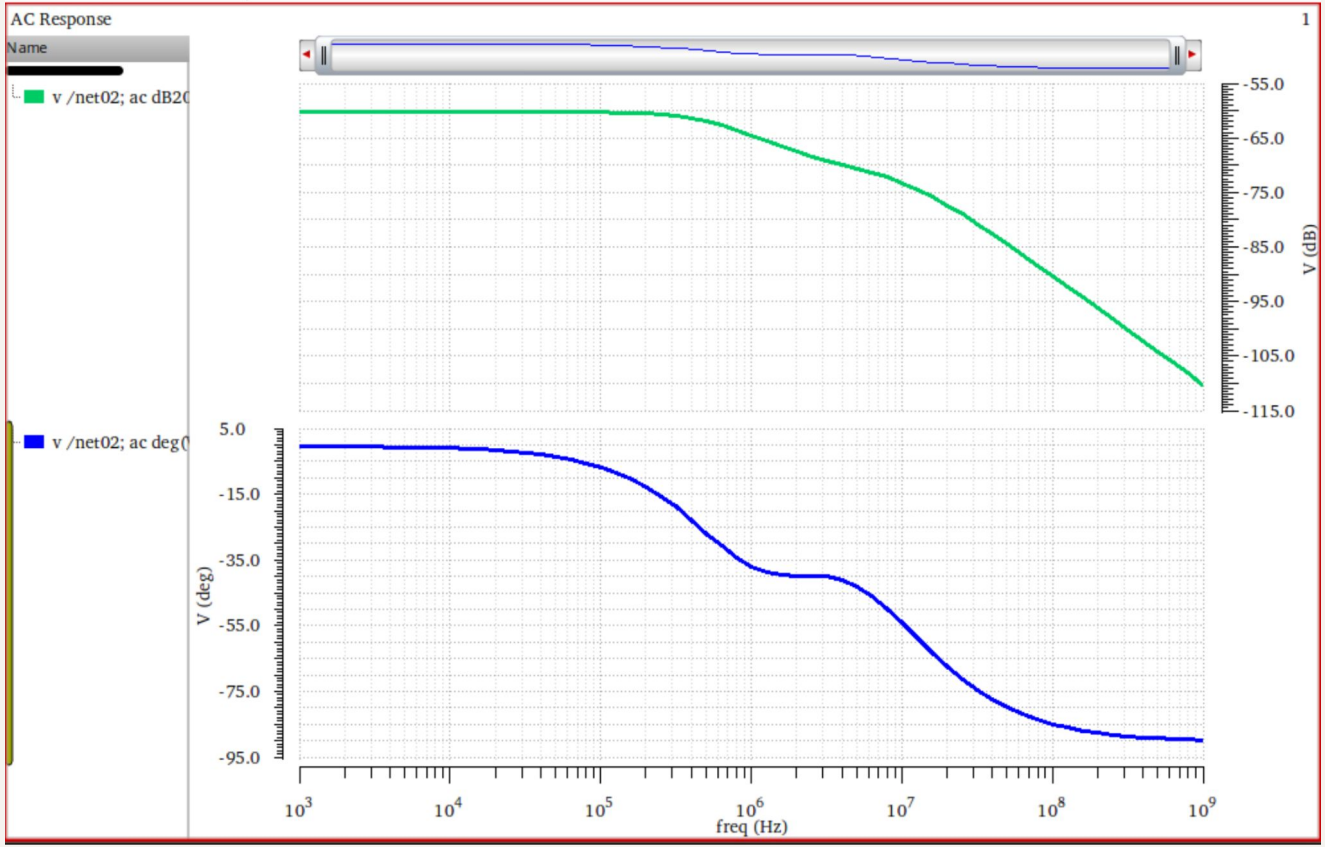
Final Demo

Test 1 - Low Pass Filter

- AC Sweep from 1K to 1G

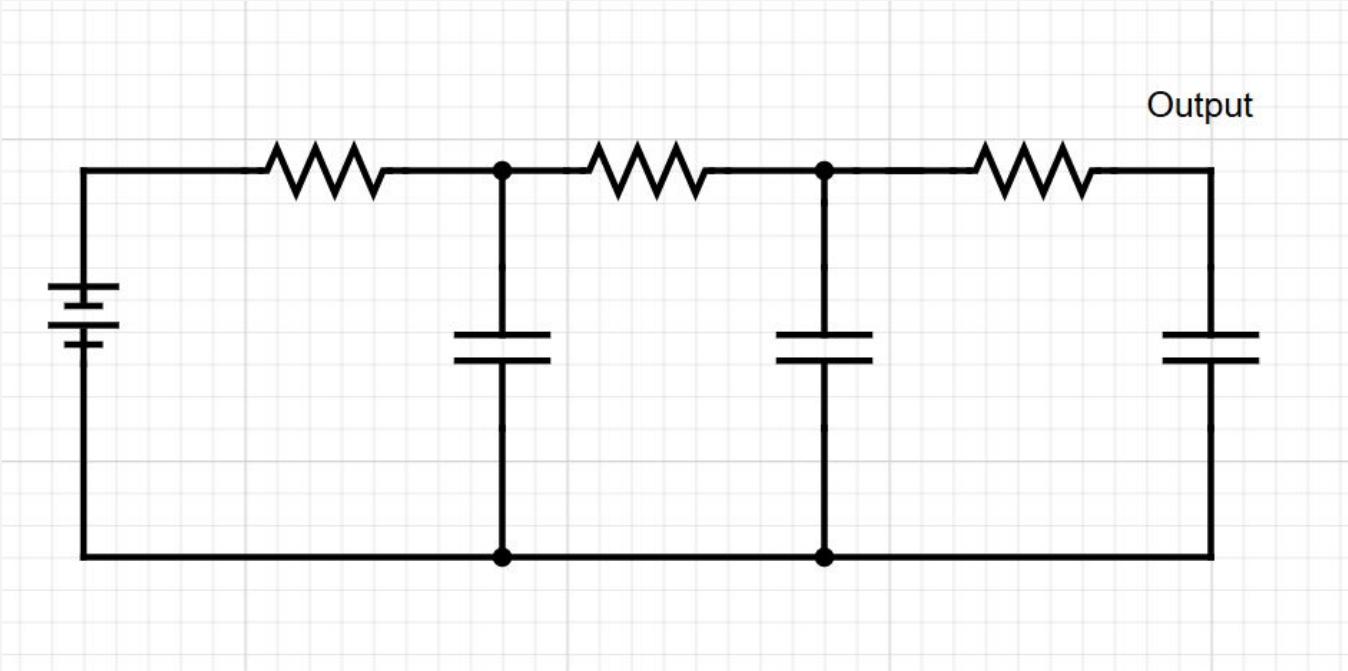


Test 1 - Low Pass Filter

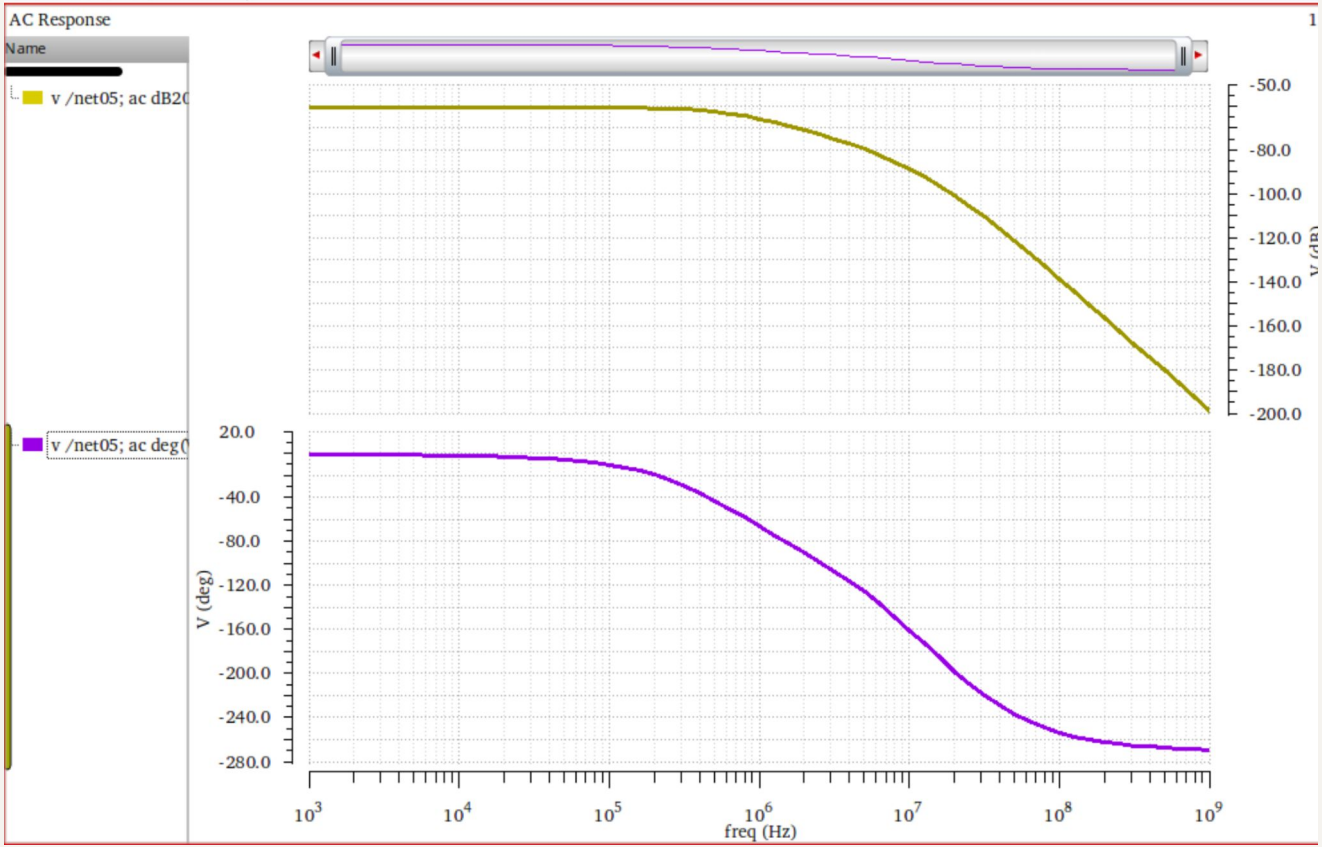


Test 2 - Low Pass Filter

- AC Sweep from 1K to 1G

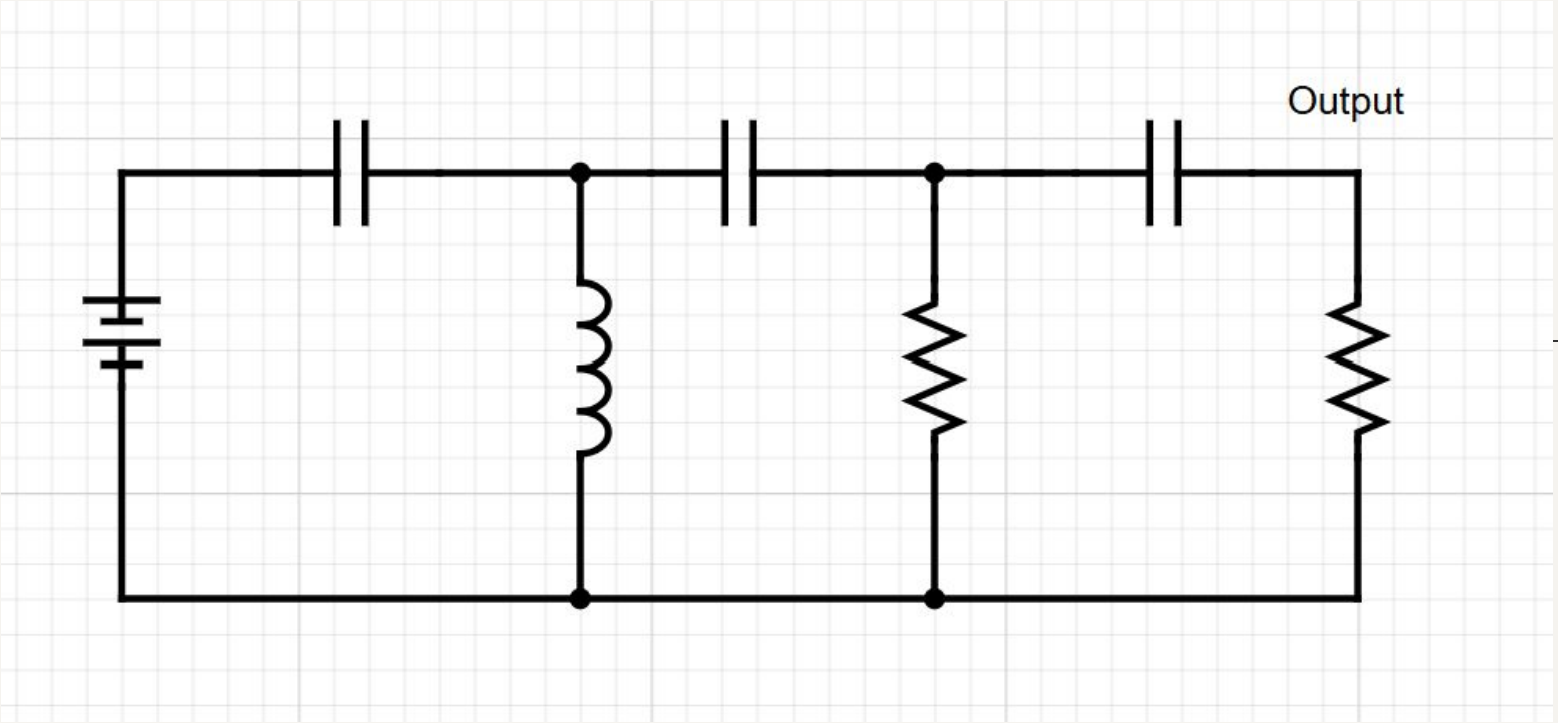


Test 2 - Low Pass Filter

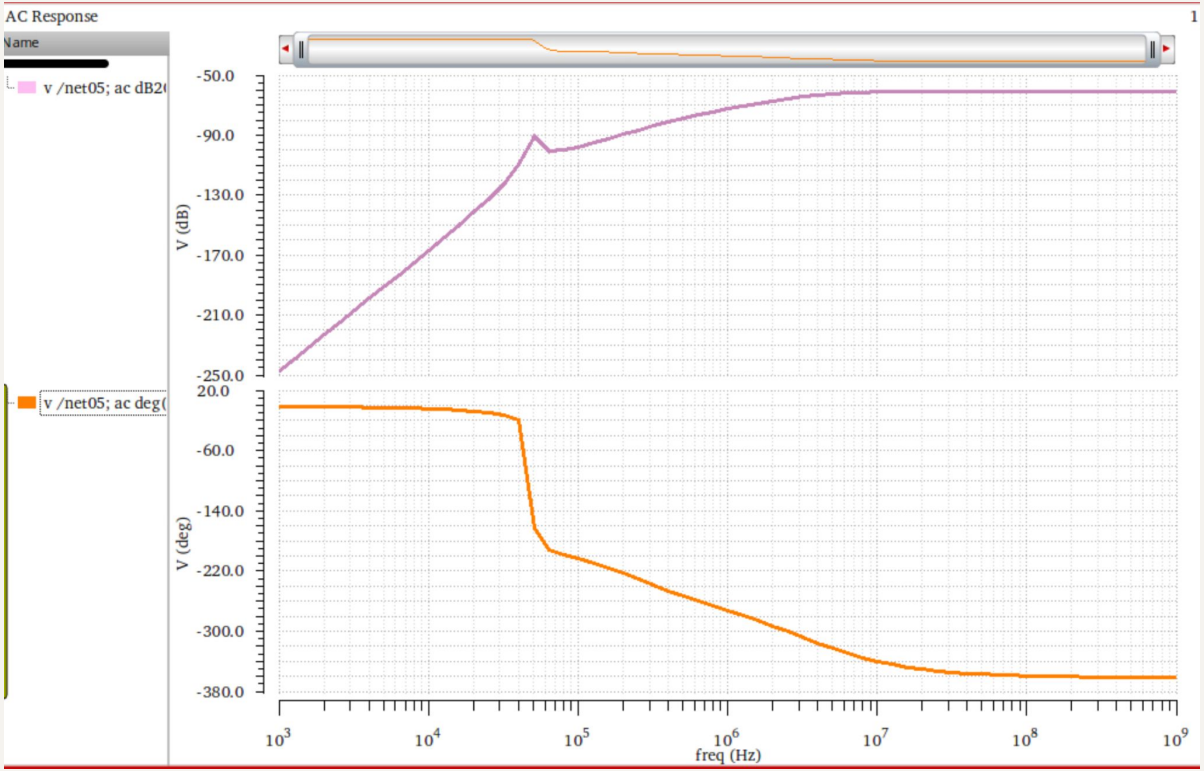


Test 3 - High Pass Filter

- AC Sweep from 1K to 1G

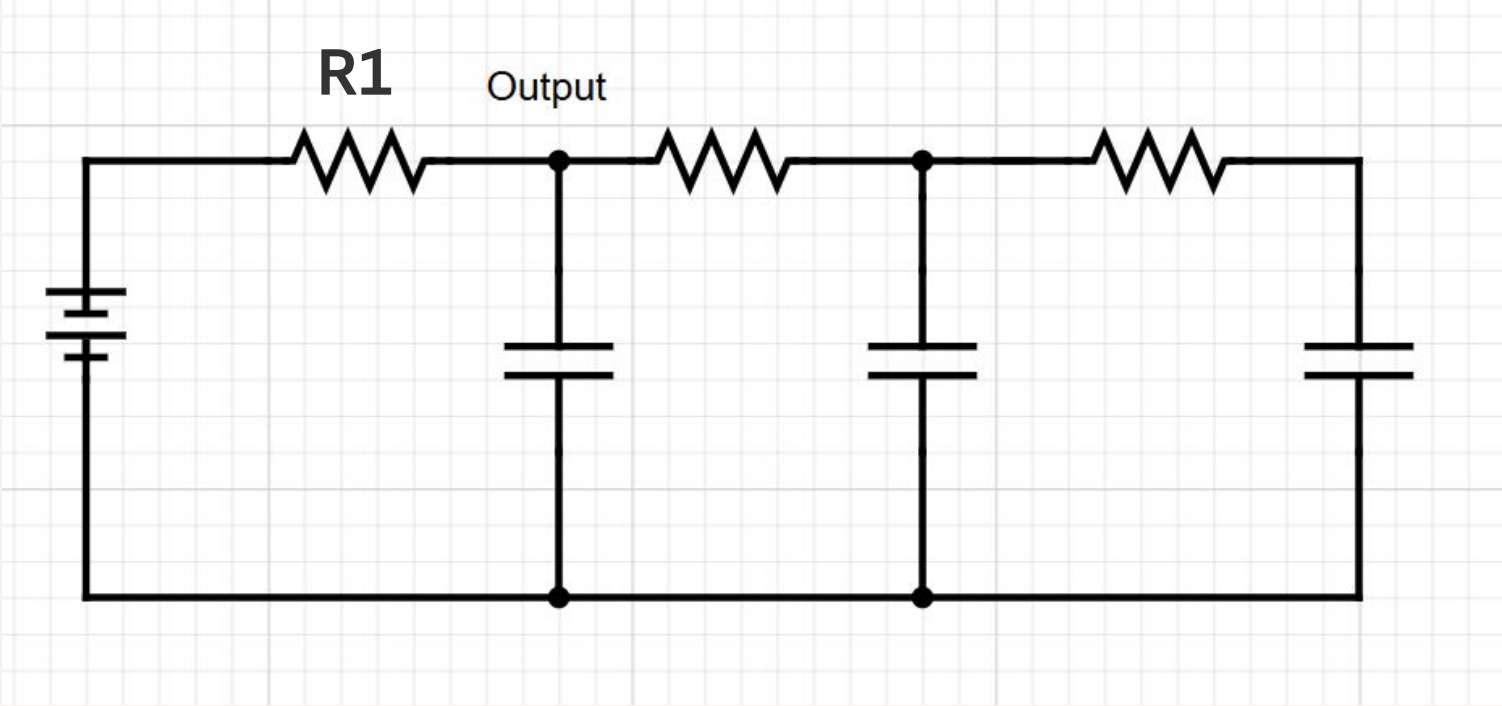


Test 3 - High Pass Filter

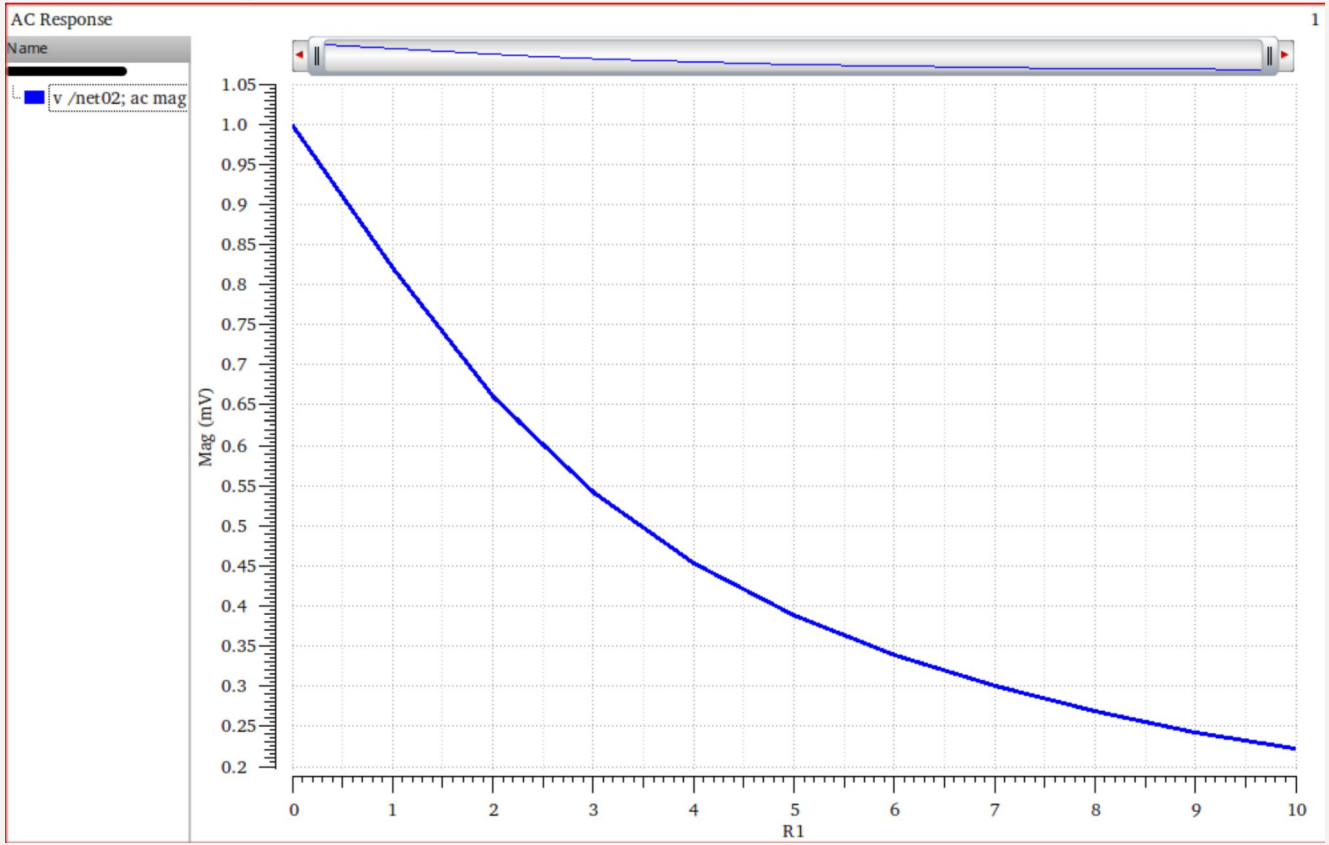


Test 4 - Parameter Sweep

- Parameter sweep R1 from 1 to 10 @ 10 MHz

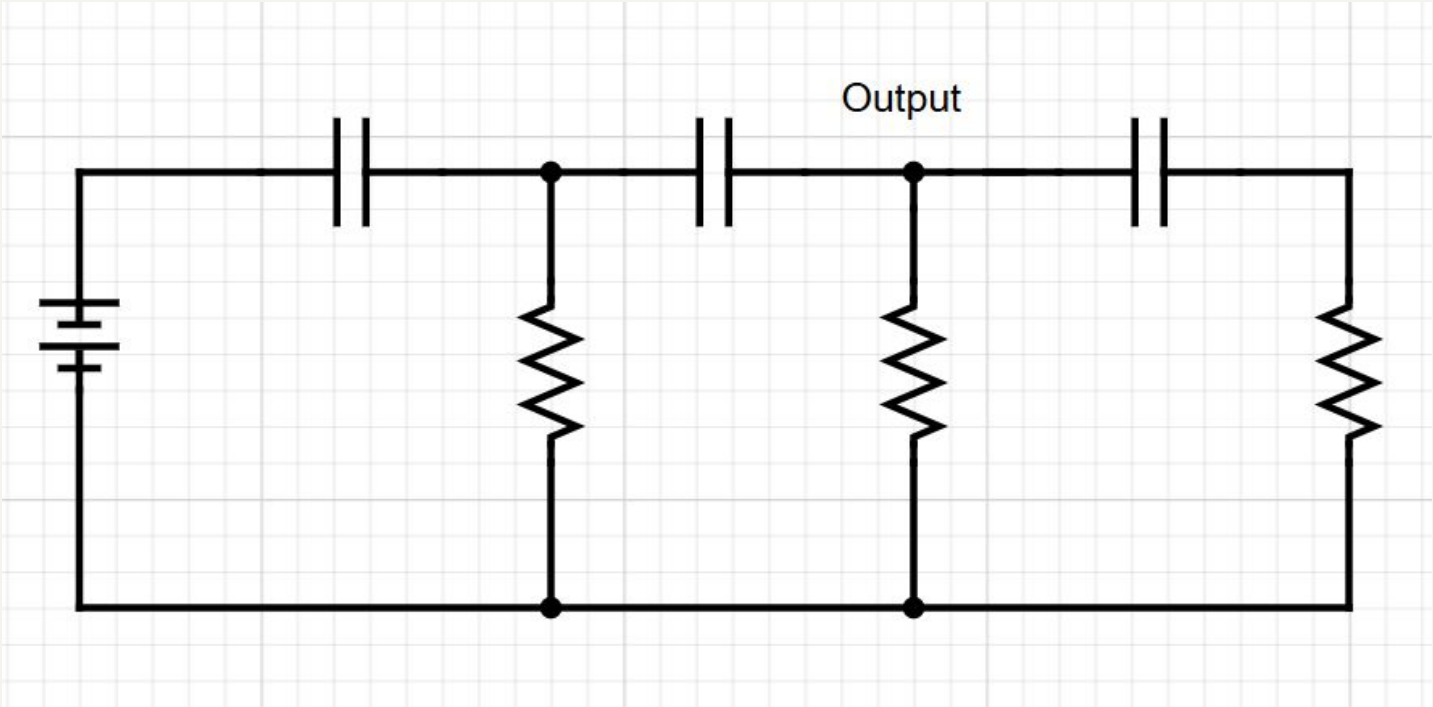


Test 4 - Parameter Sweep

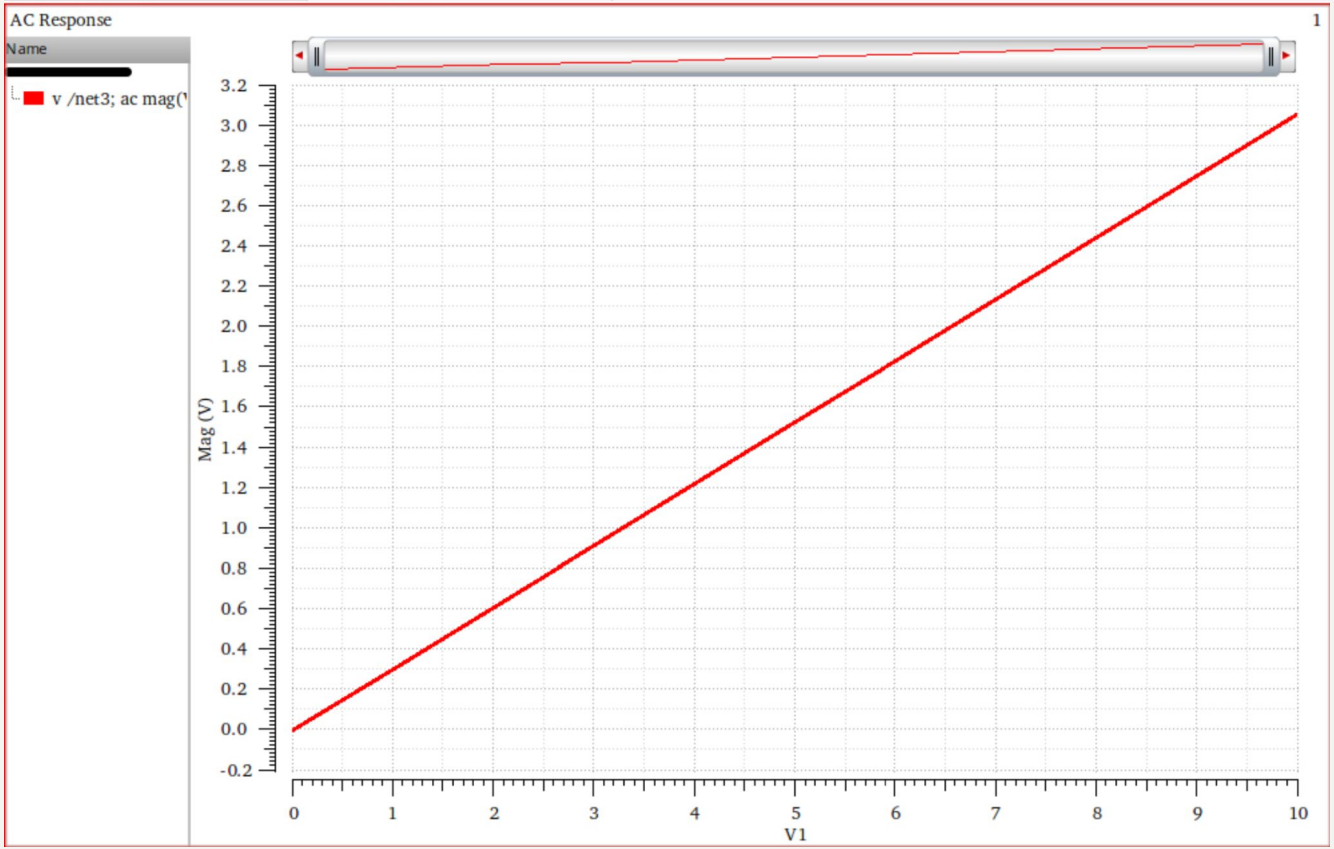


Test 5 - Voltage Sweep

- Voltage sweep from 1V to 10V



Test 5 - Voltage Sweep



Voltage Sweep Cycle Count Test:

```
SERIAL OUTPUT

MicroBlaze SPICE SFPGA_READY: Waiting for netlist data...
[SYS] Sending netlist (129 chars)  probe=V(v1)
[SYS] Transmission complete. Awaiting FPGA response...
Cycles taken for creating and saving A & b for Core 1: 1601110
Cycles taken for creating and saving A & b for Core 2: 1512616
Cycles taken for computing both Core 1 & 2: 15317
Cycles taken for creating and saving A & b for Core 1: 20862
Cycles taken for creating and saving A & b for Core 2: 20765
Cycles taken for computing both Core 1 & 2: 15311
[PARAM SWEEP] Result stream started
[PARAM SWEEP] Received 10 points for V(v1)
[SYS] Sweep complete
```

THANK YOU

OPEN QUESTIONS